

STREAM PROCESSING & ANALYTICS

Unit 4

Introduction to Apache Kafka

Why Kafka · Kafka Ecosystem · Kafka Origin · Producers & Consumers

One-night exam revision notes — crisp points, diagrams from the textbook, important items starred, and clickable links back to the exact book page.

How to read these notes	
★ IMPORTANT box	Exam-critical. Either asked in past CIE/SEE papers or a core definition. Learn these first.
Yellow highlight	Key term / keyword to underline in your answer.
→ See in book · Part 1 p.3	Blue clickable chip = opens the textbook PDF at that page (Part 1 = Ch.1 Meet Kafka, Part 2 = Ch.3 Producers + Ch.4 Consumers). Revise this page later from the book.
✍ EXAM TIP	How to frame the answer / what the examiner looks for (diagram + points).

REMEMBER Link note

- The blue links point to the book PDFs in Unitwise/Unit4/ (Part1.pdf, Part2.pdf). Open this notes PDF in **Chrome / Edge / Acrobat** on the same laptop for the links to jump to the page. If you move the folder, the links will break.

Syllabus Map & Exam Hotlist

What Unit 4 covers and what the question papers actually asked

Syllabus topic (Unit IV, 9 hrs)	Where in these notes	Book source
Why Kafka	Part A §A5	→ Ch.1 · Part 1 p.14
Kafka Ecosystem	Part A §A6 (+ pub/sub background §A1–A4)	→ Ch.1 · Part 1 p.17
Kafka Origin	Part A §A7	→ Ch.1 · Part 1 p.21
Kafka Producers	Part B (Chapter 3)	→ Ch.3 · Part 2 p.1
Kafka Consumers	Part C (Chapter 4)	→ Ch.4 · Part 2 p.45
Quick revision: tables, Q&A, flash cards	Part D	

Where Unit 4 topics appeared in past papers (taken from the CIE-1/2 2022 solutions, SEE 2022 and the Model QP in your QP folder):

Question that came	Paper	Go to
Schematically represent the Kafka producer components	CIE-1 2022	B1 (Fig 3-1)
Early (eager) vs cooperative rebalance — two differences	CIE-1 2022	C3
What happens if a broker crashes ?	CIE-1 & 2, SEE	A4 (replication) + D3
Kafka data ecosystem with a neat diagram	CIE-1, Model QP	A6 (Fig 1-9)
Producer program with high throughput + no duplicates (idempotence)	CIE-1 2022	B4, B9 code
Range assignor: 6 partitions P0–P5, 2 consumers , then add C3	CIE-1, SEE 2022	C5 worked example
Consumer that deserializes JSON to dictionary (code)	CIE-1 2022	C9 code
Consumers and consumer groups with example	SEE 2022	C1
Three ways to send messages; acks = ? + retries for at-least-once	SEE 2022 (fill-ups)	B3, B4
Commits and offsets ; two offset-commit strategies	SEE 2022	C6
Role of MirrorMaker ; reasons to choose Kafka (2 reasons)	Model QP	A4, A5
Producer–broker–consumer with a message-queue layer (diagram)	CIE-2 2022, SEE	A2 + A3
Producer + consumer implementation for a scenario (code)	CIE-2 2022	B9, C9

🔑 EXAM TIP If you only have 2 hours

- Learn: pub/sub, topic/partition/offset, broker/controller/leader, consumer group, Why-Kafka list, Fig 1-9 ecosystem.
- Learn: producer flow (Fig 3-1), 3 send methods, **acks 0/1/all**, **idempotence**, partitioner rules.
- Learn: consumer group rules (Figs 4-1 to 4-5), **eager vs cooperative**, **commit strategies**, **assignors**, **auto.offset.reset**.
- Then rehearse the two code programs in Part D.

Part A — Introduction to Kafka

Chapter 1 · Meet Kafka · Textbook Part 1 (26 pp.)

A1. Publish / Subscribe messaging

→ Book · Part 1 p.1 → Fig 1-1-1-4 · Part 1 p.3

- **Pub/Sub** = the **sender (publisher)** does **not** address a specific receiver. It **classifies** the message; **subscribers** pick the classes they want.
- A **broker** (central point where messages are published) usually sits in the middle and decouples the two sides.
- How it starts: one app pushes metrics straight to a dashboard (point-to-point). Add a storage/analysis service, more apps, polling servers ... connections multiply into a tangle = **technical debt**.
- Fix: a **single metrics pub/sub system** (one central app that receives everything and answers queries). Then logging and user-tracking teams build **their own** pub/sub systems → duplicated effort and bugs.
- Goal: **one centralised, generic system** that grows with the business → **Apache Kafka**.

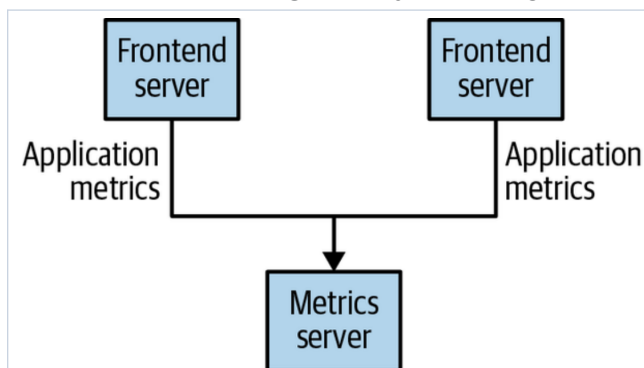


Fig 1-1. Single direct metrics publisher

→ See in book · Part 1 p.3

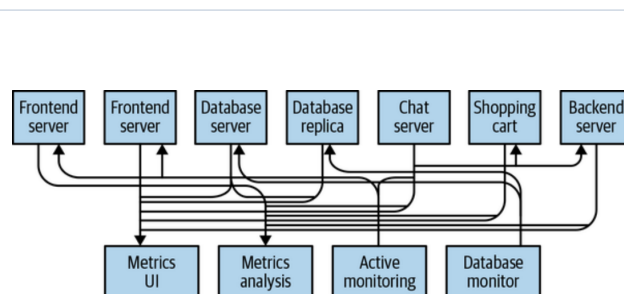


Fig 1-2. Many publishers, direct connections (messy)

→ See in book · Part 1 p.4

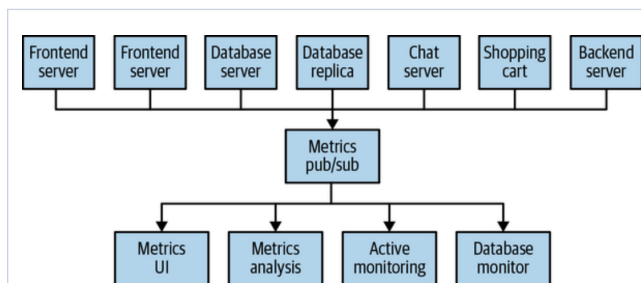


Fig 1-3. A single metrics pub/sub system

→ See in book · Part 1 p.4

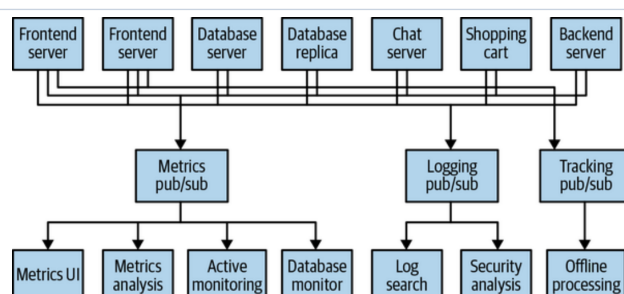


Fig 1-4. Multiple separate pub/sub systems (duplication)

→ See in book · Part 1 p.5

A2. What is Kafka? Messages, batches, schemas

→ Book · Part 1 p.5 → Messages & Batches · Part 1 p.6 → Schemas · Part 1 p.7

★ IMPORTANT Definition (write this first in any answer)

- **Apache Kafka** is a **publish/subscribe messaging system**, described as a **distributed commit log** or (more recently) a **distributed streaming platform**.
- Like a database commit log: data is stored **durably, in order** and can be **read deterministically** (replayed to rebuild state). It is also **distributed** for fault-tolerance and scaling.

Term	Meaning
Message	Unit of data in Kafka (like a row/record). To Kafka it is just a byte array ; no format/meaning is imposed.
Key	Optional metadata (also a byte array). Used to write to partitions in a controlled way: hash(key) mod #partitions → same key always lands in the same partition (as long as partition count is unchanged).
Batch	Collection of messages for the same topic + partition , written together. Fewer network round trips. Trade-off: latency vs throughput (bigger batch = more throughput, higher latency). Batches are usually compressed (saves bandwidth/storage, costs CPU).

Term	Meaning
Schema	Structure imposed on message content. JSON/XML: easy & readable but weak type handling and no version compatibility. Apache Avro (preferred): compact binary, schema kept separate from payload, no code regeneration on change, strong typing, schema evolution (forward + backward compatible).

REMEMBER Why schemas matter

- A consistent format **decouples reading from writing**: publishers can change format without coordinating with every subscriber. Schemas live in a common repository (Schema Registry, see B6).

A3. Topics, partitions, producers, consumers

→ Topics & Partitions · Part 1 p.8 → Producers & Consumers · Part 1 p.9 → Fig 1-5, 1-6 · Part 1 p.9

- Topic** = category of messages (like a **DB table / folder**). Split into **partitions**.
- Partition** = a single **append-only log**, read in order from beginning to end. **Ordering is guaranteed only within a partition, NOT across the whole topic.**
- Partitions give **redundancy + scalability**: each partition can live on a different server (horizontal scaling) and can be **replicated**.
- Stream** = usually one topic regardless of partition count (producers → consumers). Stream processing (Kafka Streams, Samza, Storm) works in **real time**, unlike offline bulk frameworks such as Hadoop.
- Producer** (publisher/writer) creates messages for a topic. Default: **balances evenly over partitions**; with a key + partitioner (hash) or a **custom partitioner** it targets specific partitions.
- Consumer** (subscriber/reader) subscribes to topics and reads each partition **in order**. It tracks progress by the **offset**.
- Offset** = ever-increasing integer metadata that Kafka adds to each message; unique per partition. Storing the next offset (in Kafka) lets a consumer **stop and restart without losing its place**.
- Consumer group** = consumers working together on a topic; **each partition is consumed by only one member** (partition ownership). Lets consumers scale horizontally; if one fails, others take over its partitions.
- Other clients: **Kafka Connect API** (data integration) and **Kafka Streams** (stream processing) — built on producers/consumers.

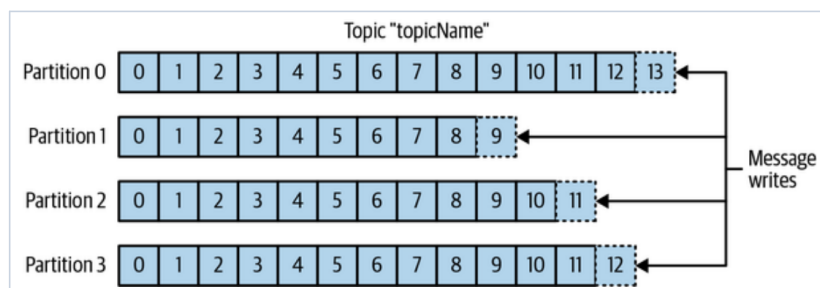


Fig 1-5. A topic with multiple partitions (writes appended to the end of each) → See in book · Part 1 p.9

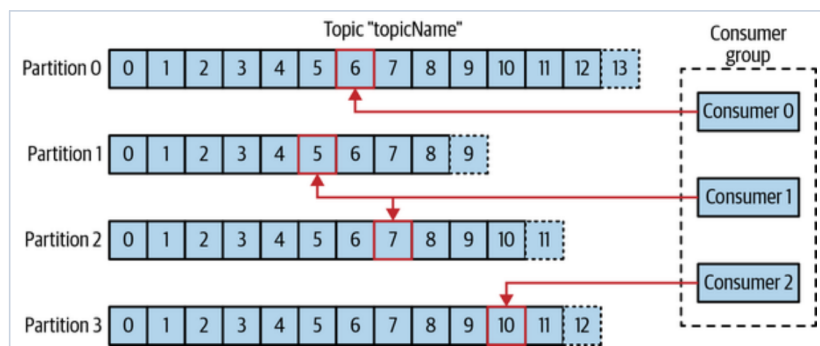


Fig 1-6. A consumer group reading from a topic (3 consumers, 4 partitions) → See in book · Part 1 p.11

A4. Brokers, clusters, retention, MirrorMaker

→ Brokers & Clusters · Part 1 p.11 → Retention · Part 1 p.12 → Multiple clusters / MirrorMaker · Part 1 p.13 → Fig 1-7, 1-8 · Part 1 p.12

- Broker** = a single Kafka server. Receives from producers, **assigns offsets, writes to disk**; serves consumer fetch requests. One broker can handle **thousands of partitions and millions of messages/sec**.

- Brokers form a **cluster**. One broker is automatically elected **cluster controller** → assigns partitions to brokers and monitors broker failures.
- A partition is owned by one broker = **leader**; copies on other brokers = **followers** (replication). **Producers must write to the leader**; consumers may read from the leader **or a follower**.
- Broker crash**: a follower (an in-sync replica) is promoted to leader; clients reconnect and consumers resume from the last committed offset.
- Retention** = durable storage for a period (e.g. **7 days**) **or** size (e.g. **1 GB** per partition); then messages expire. Configurable **per topic**.
- Log compaction**: Kafka keeps only the **last message per key** — ideal for changelog data.
- Multiple clusters** are used for: segregating data types, security isolation, and **multiple datacentres (disaster recovery)**.
- Replication inside Kafka works **only within one cluster**. Between clusters use **MirrorMaker** = simply a **Kafka consumer and producer linked by a queue**: consumes from cluster 1, produces to cluster 2.

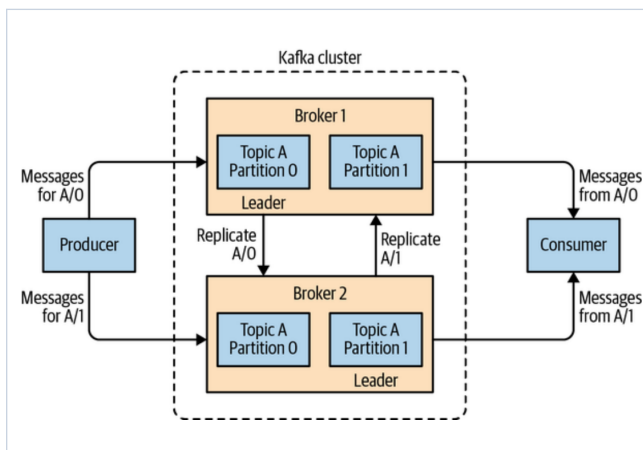


Fig 1-7. Replication of partitions in a cluster (leader + followers)
→ See in book · Part 1 p.12

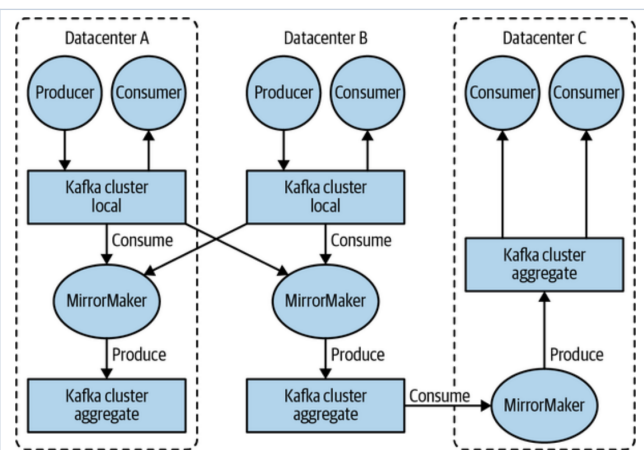


Fig 1-8. Multiple datacentres using MirrorMaker
→ See in book · Part 1 p.14

★ **IMPORTANT Model-QP question: "Role of MirrorMaker?"**

- Copies data **between Kafka clusters / datacentres** (intra-cluster replication cannot). Acts as consumer of source cluster + producer to target. Used for aggregation of local clusters into a central aggregate cluster, DR, and sharing user activity across sites.

A5. Why Kafka? (Syllabus topic)

→ Book · Part 1 p.14 pp.14–17

★ **IMPORTANT Learn all six — asked as "give any two reasons to support Kafka"**

- 1. Multiple producers** — many front-end systems write to the same topic in a common format; consumers see one consistent stream (e.g. page views from many microservices).
- 2. Multiple consumers** — many clients read the same stream **without interfering**; unlike queues where a message is gone once consumed. Consumers can share a stream in a group so each message is processed once per group.
- 3. Disk-based retention** — consumers need not be real-time; slow consumers/bursts/maintenance cause **no data loss**; restart and continue where they left off. Per-topic rules.
- 4. Scalable** — start with 1 broker → 3-broker dev cluster → tens/hundreds in production; expand **online without downtime**; higher replication factor tolerates more failures.
- 5. High performance** — producers, consumers and brokers scale out; **sub-second latency** from produce to consume under high load.
- 6. Platform features** — Kafka Connect and Kafka Streams APIs/libraries (see A6).

Feature	Traditional queue	Kafka
After a message is consumed	Gone for others	Remains (retention); every consumer group can read it
Consumer position	Broker tracks acks per message	Consumer tracks offset ; commits last processed
Scaling reads	Limited	Add consumers to a group (up to #partitions)

Feature	Traditional queue	Kafka
Storage	Transient	Durable, ordered log on disk

A6. Kafka Ecosystem (data ecosystem) & use cases

→ Platform features · Part 1 p.16 → Data Ecosystem + Fig 1-9 · Part 1 p.17 → Use cases · Part 1 p.18

- **Inputs** = applications that create data; **outputs** = metrics, reports, data products. **Loops**: read from the system, transform with other data, write back.
- Kafka is the "**circulatory system**" of the data ecosystem: carries messages between all members with **one consistent interface**. With a schema system, producers and consumers need **no tight coupling or direct connections**; components can be added/removed freely, producers need not know who consumes.
- **Kafka Connect**: pulls data from a source system into Kafka / pushes from Kafka into a sink system. **Kafka Streams**: library for scalable, fault-tolerant stream-processing apps.
- Surrounding projects: **Cruise Control**, **Kafka Monitor**, **Burrow** (LinkedIn); **ksqlDB**, **Schema Registry**, **REST proxy** (Confluent); Hadoop, Elasticsearch as downstream consumers.

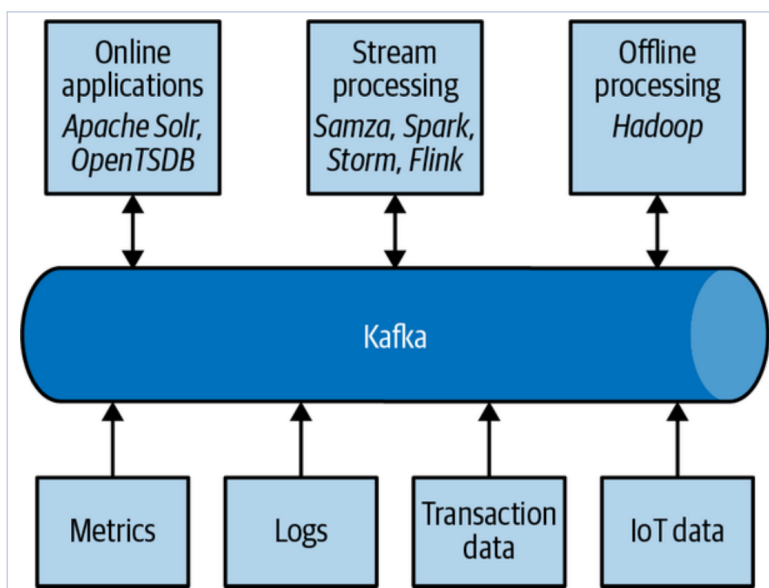


Fig 1-9. A big-data ecosystem with Kafka in the centre → See in book · Part 1 p.18

★ IMPORTANT Use cases — one line each

- **Activity tracking** (original LinkedIn use): page views, clicks, profile updates → topics → reports, ML, search updates.
- **Messaging**: apps just produce; one app formats (**decorates**), batches multiple messages into one notification, applies user preferences.
- **Metrics & logging**: many apps produce same message type; consumed by monitoring/alerting, Hadoop (long-term), Elasticsearch/security tools. Changing the destination needs no change in front ends.
- **Commit log**: publish DB changes; replicate to remote systems; consolidate changes; use **log-compacted** topics.
- **Stream processing**: like Hadoop MapReduce but **in real time** (counting metrics, partitioning, joining sources).

Producer → broker (queue) → consumer (asked in CIE-2 / SEE): the producer sends a message to a broker; the broker stores it in a topic partition (the queue / log); the consumer pulls it. This **message-queuing tier** decouples producers from consumers, buffers bursts (back-pressure), and lets each side fail or scale independently. Draw Fig 1-3 / 1-6 combined.

A7. Kafka Origin (LinkedIn)

→ Origin · Part 1 p.21 → Birth of Kafka · Part 1 p.23 → Open source / Commercial · Part 1 p.24

LinkedIn had two broken systems	Faults
Metrics system (CPU, app performance, request tracing)	Polling-based collection; large intervals between metrics; owners could not manage own metrics; high-touch (manual work); inconsistent metric names .
User-activity tracking (HTTP service, XML batches from front ends)	Inconsistent, costly-to-parse XML; changing tracked activity needed coordinated work; broke on schema changes; hourly batch → no real time.

- The two could **not share one backend** (polling vs push, clunky vs fragile) although correlating them was desirable (a drop in some user activity can reveal an app problem — but hours of delay hid it).
- Off-the-shelf options were tried: **ActiveMQ** prototype could not handle the scale and its brokers **paused**, backing up clients → decision to build **custom infrastructure**.
- Team led by **Jay Kreps** (earlier built Voldemort), with **Neha Narkhede** and later **Jun Rao**.

★ **IMPORTANT Kafka's four design goals (frequently asked)**

- **Decouple** producers and consumers using a **push-pull** model.
- Provide **persistence** of message data so **multiple consumers** can read it.
- Optimise for **high throughput**.
- Allow **horizontal scaling** as data streams grow.
- Result: messaging-style interface + **log-aggregation-style storage**; with **Avro** it handled metrics + activity tracking at **billions of messages/day** (LinkedIn: >7 trillion msgs produced, >5 PB consumed daily, as of Feb 2020).

When	Milestone
Late 2010	Released as open source on GitHub
July 2011	Accepted as Apache Software Foundation incubator project
Oct 2012	Graduated from incubator
Fall 2014	Kreps, Narkhede, Rao leave LinkedIn to found Confluent (support, training, managed Kafka on GCP/AWS/Azure; ksqldb, schema registry, REST proxy)
2016 →	Kafka Summit conferences (US & London)

Name: chosen by Jay Kreps because Kafka is optimised for *writing*, so a writer's name (Franz Kafka) seemed fitting — "basically there is not much of a relationship". Users today include Netflix and Uber. [→ Name · Part 1 p.25](#)

Part B — Kafka Producers

Chapter 3 · Writing Messages to Kafka · Textbook Part 2, pp.1–43

B1. Producer overview and components

→ Overview · Part 2 p.2 → Fig 3-1 flow · Part 2 p.4

- Every use of Kafka = a **producer** writing, a **consumer** reading, or an app doing both (e.g. credit-card system: store → Kafka → rules engine → Kafka → store/DB).
- Requirements differ: **can we lose messages? duplicates OK? latency / throughput limits?** Credit-card: never lose/duplicate, <500 ms, ~1 M msgs/s. Click-stream: some loss/duplicates OK, latency can be higher.
- Third-party clients exist via Kafka's **binary wire protocol** (Python, C++, Go ...).

★ IMPORTANT Producer data flow (draw Fig 3-1) — CIE-1 2022

1. Create a **ProducerRecord**: **topic + value** are mandatory; *[partition]*, *[key]*, *[timestamp]*, *[headers]* optional.
2. **send()** → **Serializer** converts key and value to **byte arrays**.
3. If no partition given → **Partitioner** picks one (usually from the key).
4. Record is added to a **batch** for that topic-partition (in an in-memory buffer).
5. A **separate sender thread** ships batches to the right brokers.
6. Broker replies: success → **RecordMetadata** (topic, partition, offset); failure → error → **retry** if possible, else exception.

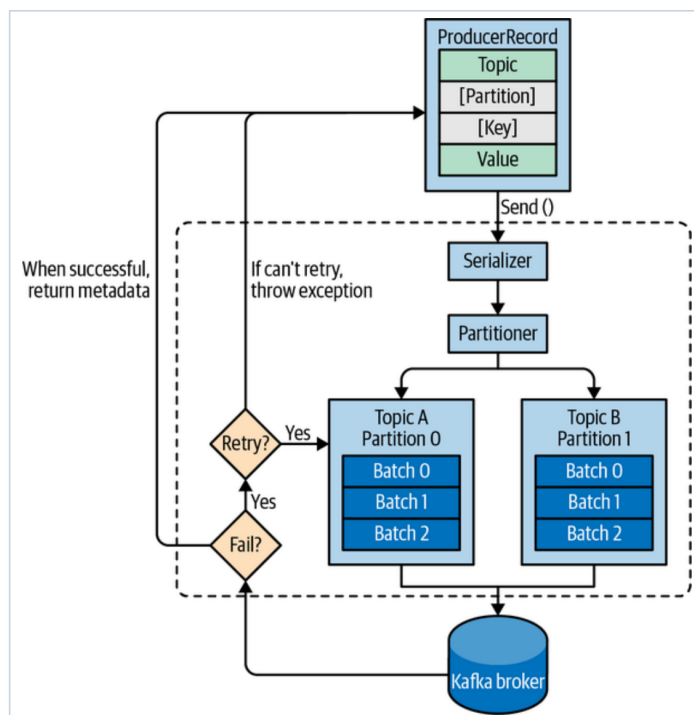


Fig 3-1. High-level overview of Kafka producer components → See in book · Part 2 p.4

B2. Constructing a producer — mandatory properties

→ Book · Part 2 p.5 pp.5–7

Property	Purpose
bootstrap.servers	List of host:port of brokers for the initial connection . Need not list all brokers (more info is fetched after connecting); give at least two in case one is down.
key.serializer	Class implementing Serializer that turns the key object into bytes. Required even if you only send values (use Void type + VoidSerializer). Built-ins: ByteArray , String , Integer ...
value.serializer	Same, for the value.

```
Properties kafkaProps = new Properties();
kafkaProps.put("bootstrap.servers", "broker1:9092,broker2:9092");
kafkaProps.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
kafkaProps.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
producer = new KafkaProducer<String, String>(kafkaProps);
```

B3. Three ways to send a message

→ Book · Part 2 p.7 → Fire & forget · Part 2 p.8 → Sync · Part 2 p.9 → Async + callback · Part 2 p.11

★ **IMPORTANT SEE 2022 fill-up: "The three primary methods of sending messages are ____"**

- **1. Fire-and-forget** — send and ignore result. Fastest; message can be **lost silently** on non-retriable error/timeout.
- **2. Synchronous send** — `producer.send(record).get()` waits on the **Future**; catches errors and gets RecordMetadata. **Poor throughput** (thread idles 2 ms–seconds per request) so rarely used in production.
- **3. Asynchronous send** — `send(record, callback); onCompletion(RecordMetadata, Exception)` fires on response. High throughput + error handling. **Recommended.**
- Technically the producer is **always asynchronous**; `get()` just waits for the Future.
- Async example: RTT 10 ms → 100 sync sends ≈ **1 s**; async sends barely take time.
- A producer object is **thread-safe** and can be shared by many threads.
- **Retriable errors** (connection lost, *not leader for partition*) → producer retries automatically. **Non-retriable** (e.g. *message size too large*) → exception returned immediately.
- Exceptions before sending: **SerializationException**, **BufferExhaustedException/TimeoutException** (buffer full), **InterruptedException**.
- Callbacks run in the producer's **main thread** (order preserved per partition) → keep them **fast / non-blocking**.

B4. Important producer configuration

→ Configs start · Part 2 p.12 → acks · Part 2 p.13 → Delivery time · Part 2 p.15 → linger/batch · Part 2 p.19 → idempotence · Part 2 p.22

★ **IMPORTANT acks — how many replicas must receive the record before the write is "successful"**

- **acks=0**: producer does **not wait**. Highest throughput, **may lose data silently**.
- **acks=1** (default before Kafka 3.0): success once the **leader** has it. Loss possible if leader crashes before replicating.
- **acks=all**: success once **all in-sync replicas** have it. **Safest**, highest latency.
- End-to-end latency is the **same** for all three, because consumers can only read records already written to all in-sync replicas. (Reliability trades off only *producer* latency.)
- **At-least-once delivery**: `acks=all + retries / large delivery.timeout.ms`. Add `enable.idempotence=true` to remove duplicates (SEE 2022 fill-up).

Config	What it does	Notes
client.id	Logical name of the client app	Used in logs, metrics, quotas ; makes troubleshooting easy
max.block.ms	How long <code>send()</code> / <code>partitionsFor()</code> may block (buffer full / no metadata)	Then timeout exception
delivery.timeout.ms	Total time from batch-ready until success/give-up including retries	Must be > <code>linger.ms + request.timeout.ms</code> . Prefer this over retries (e.g. 120 s for ~30 s leader election)
request.timeout.ms	Wait per request for a broker reply	Excludes retries
retries / retry.backoff.ms	Number of retries / wait between (default 100 ms)	Not recommended to tune; <code>retries=0</code> is the only way to disable retries
linger.ms	Wait for more messages before sending the batch	Default 0. >0 → slightly more latency, much higher throughput , better compression
batch.size	Memory (bytes , not messages) per batch	Sends half-full batches too; big = more memory, small = overhead
buffer.memory	Total memory buffering unsent records	When full, <code>send()</code> blocks <code>max.block.ms</code> then throws
compression.type	none (default), snappy, gzip, lz4, zstd	Snappy : low CPU, good speed. Gzip : more CPU, best ratio (bandwidth-limited)

Config	What it does	Notes
max.in.flight.requests.per.connection	Unacknowledged batches allowed at once	Default 5; more = throughput & memory
max.request.size	Caps size of a request / largest message (default 1 MB)	Match broker's <code>message.max.bytes</code>
send/receive.buffer.bytes	TCP socket buffers (-1 = OS default)	Increase for cross-datacentre links
enable.idempotence	Producer adds a sequence number to each record; broker rejects duplicates	Needs acks=all, retries>0, max.in.flight ≤ 5; keeps order with up to 5 in flight

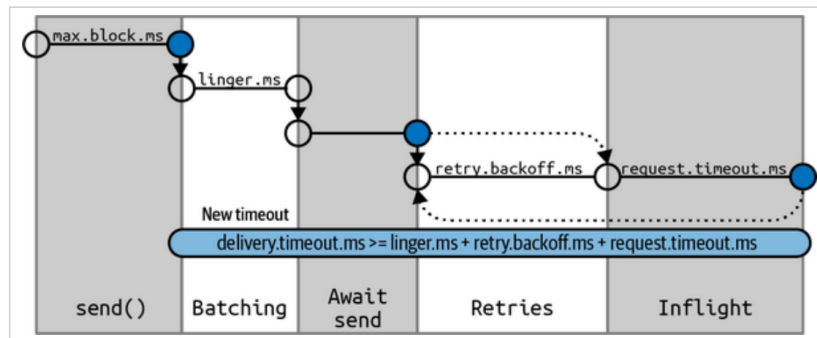


Fig 3-2. Delivery-time breakdown inside the producer (send() → batch → callback) → See in book · Part 2 p.16

★ IMPORTANT Ordering & duplicates (frequently confused)

- Kafka preserves order **within a partition**. With **retries > 0** and **max.in.flight > 1**, batch 1 may fail, batch 2 succeed, batch 1 retried → **order reversed**.
- **Duplicate case**: broker writes + replicates the record but crashes before replying; the producer times out and retries to the new leader → two copies.
- **Fix**: `enable.idempotence=true` → sequence numbers; broker rejects the 2nd copy (harmless `DuplicateSequenceException`) and order is guaranteed with ≤5 in-flight. This is a building block of **exactly-once** (Kafka ≥ 0.11).

B5. Serializers

→ Serializers · Part 2 p.23 → Custom serializer code · Part 2 p.24

- Brokers store **bytes**; serializers convert objects → bytes. Built-ins: String, Integer, ByteArray ...
- **Custom serializer** (implements `Serializer<T>`: configure / serialize / close) is **fragile**: changing a field type or adding a field breaks old vs new messages; debugging means comparing raw bytes; all teams must change code simultaneously.
- → Use a generic library: **JSON, Avro, Thrift, Protobuf** (Avro most common with Kafka).

B6. Avro + Schema Registry

→ Avro · Part 2 p.26 → Schema Registry + Fig 3-3 · Part 2 p.29 → Avro producer code · Part 2 p.30

- **Avro**: language-neutral serialization format (created by **Doug Cutting**). Schema written in **JSON**; data usually **binary**.
- Big benefit: writer can move to a **new, compatible schema** and readers keep working with **no change**. Example: field `faxNumber` replaced by `email` (both with default null) → old readers get **null** for email, new readers get **null** for fax; no exceptions.
- Two caveats: schemas must be **compatible**; the deserializer needs **access to the writing schema**.
- Storing the whole schema in every record would more than **double** record size → use a **Schema Registry** (not part of Apache Kafka; e.g. Confluent): store all schemas there, put only the **schema ID** in each record. Serializer registers/fetches it; the deserializer looks it up by ID.
- Producer config: `KafkaAvroSerializer` for key/value + `schema.registry.url`. Use generated Avro classes (not POJOs) or **GenericRecord** + schema.

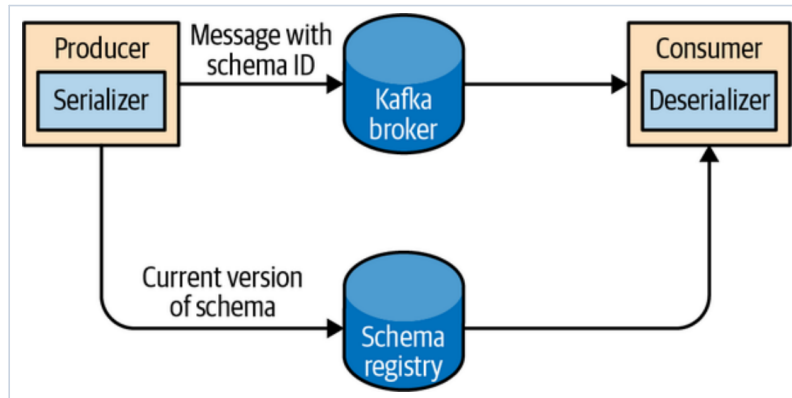


Fig 3-3. Serialization and deserialization of Avro records using the Schema Registry → See in book · Part 2 p.30

B7. Partitions & partitioners

→ Partitions · Part 2 p.33 → Sticky null-key · Part 2 p.34 → Custom partitioner code · Part 2 p.35

★ IMPORTANT Partitioner rules

- **Key = null** (default partitioner): spreads records over available partitions; since Kafka 2.4 the round-robin is **sticky** — fills one batch for a partition before moving on → fewer requests, lower latency & CPU.
- **Key present**: Kafka **hashes the key** (own hash, murmur2) and maps it onto **all** partitions (not just available ones) → **same key** → **same partition**. If that partition is down you may get an error.
- **Mapping holds only while partition count is constant**. Adding partitions later breaks it → create topics with enough partitions and never add.
- Alternatives: **RoundRobinPartitioner**, **UniformStickyPartitioner** (even load even with keys / skewed keys).
- **Custom partitioner** implements **Partitioner** (configure/partition/close). Example "Banana": a customer with > 10% of traffic is sent to the **last partition**; the rest are hashed over the others.

B8. Headers, interceptors, quotas

→ Headers · Part 2 p.37 → Interceptors · Part 2 p.37 → Quotas & throttling · Part 2 p.40

Feature	Key points
Headers	Ordered key/value pairs (key = String, value = any serialized object) holding metadata without touching the key/value . Uses: lineage (source of data), routing/tracing without parsing (even if encrypted). <code>record.headers().add("privacy-level", bytes)</code>
Interceptors	ProducerInterceptor changes behaviour without modifying app code (just add the jar + <code>interceptor.classes</code> config). <code>onSend(record)</code> — before serialization, may modify and must return a record; <code>onAcknowledgement(metadata, exception)</code> — on broker ack, cannot modify. Uses: monitoring/tracing, standard headers, redacting sensitive data .
Quotas	Brokers limit rates. 3 types : produce , consume (bytes/sec) and request (% of broker processing time). Defaults e.g. <code>quota.producer.default=2M</code> ; per-client override; dynamic via <code>kafka-configs.sh/AdminClient</code> (static ones need broker restart). On breach the broker throttles (delays responses and mutes the channel). Visible via <code>produce-throttle-time-avg/max</code> metrics.

REMEMBER Warning from the book

- If an async producer keeps sending faster than the broker accepts, records queue in client memory → buffer fills → `send()` blocks → **TimeoutException** (or batches expire after `delivery.timeout.ms`). Plan and monitor broker capacity.

B9. Exam-ready producer program (Python)

CIE-1 2022 asked a producer for an orders topic with **high throughput and no duplicate orders**. Same idea in confluent-kafka:

```

from confluent_kafka import Producer
import json

conf = {
    "bootstrap.servers": "localhost:9092", # initial broker list
    "acks": "all", # wait for all in-sync replicas
    "enable.idempotence": True, # no duplicates, keeps order
    "linger.ms": 20, # small wait -> bigger batches
    "batch.size": 65536, # throughput
    "compression.type": "snappy", # less network, little CPU
}
producer = Producer(conf)
TOPIC = "orders"

def on_delivery(err, msg): # asynchronous callback
    if err is not None:
        print("Delivery failed:", err)
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}")

order = {"order_id": 101, "item": "laptop", "qty": 1}
producer.produce(TOPIC, key=str(order["order_id"]), # same key -> same partition
                 value=json.dumps(order), callback=on_delivery)
producer.poll(0) # serve delivery callbacks
producer.flush() # block until all messages are sent

```

EXAM TIP Say in the answer

- acks=all + idempotence → no loss, no duplicates; linger.ms + batch.size + compression → throughput; key → per-key ordering; callback → async error handling.

Part C — Kafka Consumers

Chapter 4 · Reading Data from Kafka · Textbook Part 2, pp.45–98

C1. Consumers and consumer groups

→ Book · Part 2 p.45 → Fig 4-1 → 4-5 · Part 2 p.47

- A single consumer can **fall behind** if producers write faster than it processes → need to **scale consumption** by splitting a topic among several consumers.
- Consumers normally belong to a **consumer group** (group.id). In a group, **each consumer reads a different subset of the topic's partitions**.

★ IMPORTANT The four rules (draw Figs 4-1 to 4-5)

- **1 consumer** in group G1, topic T1 with 4 partitions → it gets **all 4** partitions (Fig 4-1).
- **2 consumers** → 2 partitions each (Fig 4-2). **4 consumers** → 1 each (Fig 4-3).
- **More consumers than partitions** → the extra consumers are **idle** (Fig 4-4). Never use more consumers than partitions → create topics with many partitions up front.
- **Each application needs its own consumer group** to receive **all** messages (Fig 4-5): G2 gets everything independently of G1. Within one group each partition has **one owner**.
- To **scale** an app: add consumers to the same group. To **add another app**: new group. Kafka scales to many groups without performance loss.

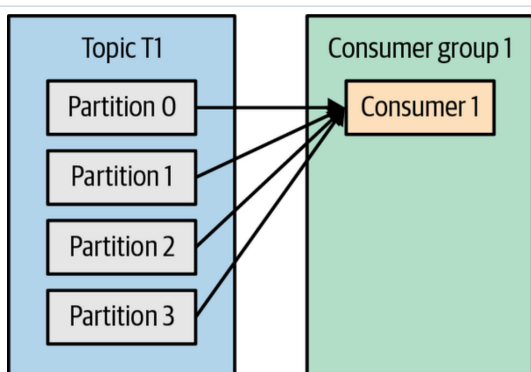


Fig 4-1. One consumer, four partitions
→ See in book · Part 2 p.47

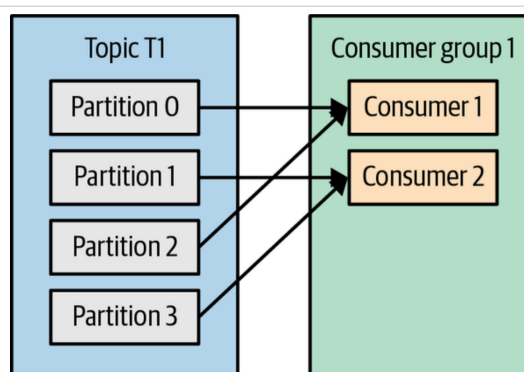


Fig 4-2. Four partitions split over two consumers
→ See in book · Part 2 p.48

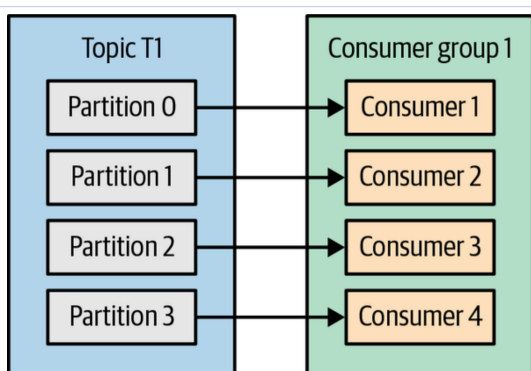


Fig 4-3. Four consumers, one partition each
→ See in book · Part 2 p.49

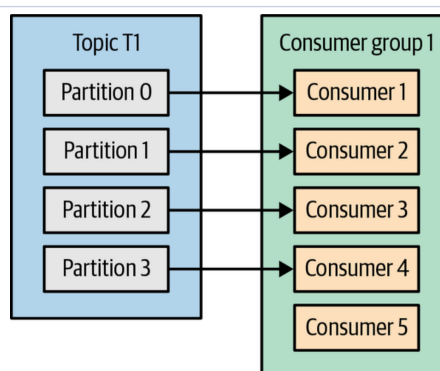


Fig 4-4. More consumers than partitions → idle consumer
→ See in book · Part 2 p.50

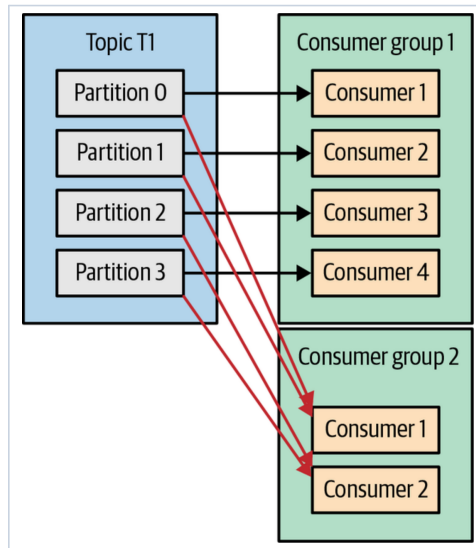


Fig 4-5. Adding a new consumer group — both groups receive all messages → See in book · Part 2 p.52

C2. Consumer-group rebalance

→ Rebalance · Part 2 p.53 → Heartbeats / coordinator · Part 2 p.55 → Static membership · Part 2 p.56

- **Rebalance** = moving partition ownership between consumers. Triggers: **new consumer joins**, consumer **shuts down / crashes**, or the topic changes (e.g. **new partitions** added).
- Good: gives **high availability and scalability**. Bad: during a rebalance consumption pauses.

	Eager (stop-the-world) rebalance	Cooperative (incremental) rebalance
What happens	All consumers stop, revoke all partitions , rejoin, receive a brand-new assignment	Only a small subset of partitions moves; others keep being consumed
Phases	2: (1) everyone gives up partitions, (2) new assignment	2 or more: leader announces which partitions will be lost, those are released, then reassigned (may take a few iterations)
Impact	Short window of unavailability of the whole group ; grows with group size	No complete "stop the world"; matters most for large groups
Assignor	Range, RoundRobin, Sticky	CooperativeStickyAssignor

CIE-1 2022: "two differences between early and cooperative rebalance" — use rows 1 & 3 (what is revoked; availability impact).

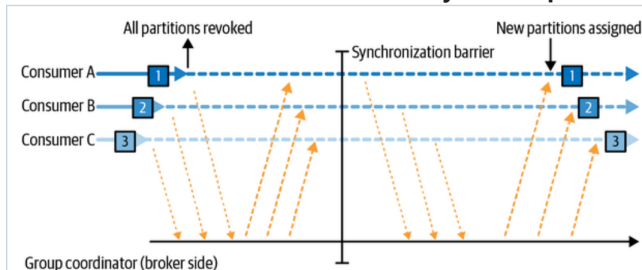


Fig 4-6. Eager: revoke all, pause, reassign

→ See in book · Part 2 p.54

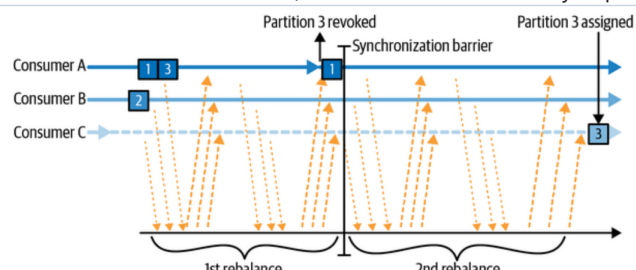


Fig 4-7. Cooperative: pause only moved partitions

→ See in book · Part 2 p.55

★ IMPORTANT Heartbeats, group coordinator, group leader

- Consumers send **heartbeats** (background thread) to a broker acting as the **group coordinator**. Heartbeats on time = alive.
- No heartbeat for `session.timeout.ms` → coordinator declares it dead → **rebalance**. A clean `close()` notifies the coordinator → immediate rebalance (smaller gap).
- **Join process**: consumer sends **JoinGroup** to the coordinator. The **first** consumer to join becomes the **group leader**; it gets the member list and, using a **PartitionAssignor**, decides who gets which partitions, then sends the plan to the coordinator which tells each member (each consumer sees only its own assignment).
- **Static membership**: set a unique `group.instance.id`. A restarting static member keeps its **same partitions without a rebalance** (good for local state/cache). Cost: partitions sit idle until it returns or `session.timeout.ms` expires; duplicate instance id → error.

C3. Creating a consumer, subscribing, the poll loop

→ Creating · Part 2 p.58 → Subscribe · Part 2 p.59 → Poll loop · Part 2 p.60 → Thread safety · Part 2 p.62

Mandatory / common property	Purpose
bootstrap.servers	Initial connection string to the cluster (same as producer)
key.deserializer, value.deserializer	Classes that turn byte arrays back to objects (mirror of serializers)
group.id	Consumer group name (not strictly mandatory but used almost always)

- `consumer.subscribe(Collections.singletonList("topic"))`; or a **regex** `subscribe(Pattern.compile("test.*"))` — a new matching topic triggers a rebalance. Regex filtering is **client-side** (costly with ~30,000+ partitions; needs describe permission on all topics).
- **Poll loop** is the heart of the Consumer API: `while(true){ records = consumer.poll(Duration.ofMillis(100)); for(record : records){ process } }`.

★ IMPORTANT "Consumers must keep polling or they are considered dead"

- The **first poll()** finds the GroupCoordinator, joins the group and gets the partition assignment. Rebalances and callbacks are handled **inside poll()**.
- If `poll()` is not called for longer than `max.poll.interval.ms` (default 5 min) the consumer is evicted and its partitions reassigned.
- Each ConsumerRecord holds **topic, partition, offset, key, value**. `poll(timeout)` = how long to block when no data is buffered.
- **Thread safety**: one consumer per thread. Cannot share a consumer across threads. Run multiple consumers via `ExecutorService`, or one consumer + a worker-thread queue. `wakeup()` is the only method safe to call from another thread.

C4. Important consumer configuration

→ Configs · Part 2 p.63 → session / heartbeat · Part 2 p.66 → auto.offset.reset · Part 2 p.68 → enable.auto.commit · Part 2 p.69

Config	Meaning / default	Why it matters
fetch.min.bytes	Min data broker returns per fetch (default 1 B)	Raise to cut CPU/broker load on low-traffic topics (adds latency)
fetch.max.wait.ms	Max wait for fetch.min.bytes (default 500 ms)	Caps added latency. Broker answers when min bytes OR max wait is reached first
fetch.max.bytes	Max bytes per poll (default 50 MB)	Limits consumer memory
max.poll.records	Max number of records per poll()	Controls processing time per loop
max.partition.fetch.bytes	Max bytes per partition (default 1 MB)	Book recommends fetch.max.bytes instead
session.timeout.ms	Time without heartbeat before dead (default 10 s)	Lower = faster failure detection but spurious rebalances
heartbeat.interval.ms	How often heartbeats are sent	Must be < session timeout, usually 1/3 of it
max.poll.interval.ms	Max gap between polls (default 5 min)	Catches a hung main thread even if heartbeats continue
default.api.timeout.ms	Timeout for most API calls (default 1 min)	<code>poll()</code> always takes an explicit timeout
request.timeout.ms	Wait for broker reply (default 30 s)	Do not lower (avoid reconnect overhead)
auto.offset.reset	What to do with no valid committed offset : latest (default), earliest , none (throw)	earliest = read from the beginning; latest = only new records
enable.auto.commit	Auto-commit offsets (default true); auto.commit.interval.ms = 5 s	Set false for manual control (fewer duplicates / no loss)
partition.assignment.strategy	Range (default), RoundRobin, Sticky, CooperativeSticky	See C5
client.rack	Zone of the consumer	Fetch from nearest replica (needs RackAwareReplicaSelector)
group.instance.id	Unique id for static membership	Avoids rebalance on restart
offsets.retention.minutes	(Broker) offsets of an empty group kept 7 days	After that the group behaves like new

C5. Partition assignment strategies + worked example

→ Assignors · Part 2 p.69

Assignor	How it works	Remarks
Range (default)	For each topic separately gives each consumer a consecutive range of partitions	If partitions don't divide evenly the first consumers get more ; bias accumulates over many topics
RoundRobin	Takes all partitions of all topics and deals them out one by one	Counts differ by at most 1 when all consumers subscribe to the same topics
Sticky	As balanced as possible and on rebalance keeps as many existing assignments as possible	Fewer partition moves; better balance than RoundRobin when subscriptions differ
CooperativeSticky	Same as Sticky but supports cooperative rebalance	Consumers keep consuming non-moved partitions

★ IMPORTANT Worked example — CIE-1 2022 & SEE 2022: topic Sales_data, 6 partitions P0–P5, group with C1, C2

- (i) **RangeAssignor with 2 consumers** ($6 / 2 = 3$ each, contiguous): **C1 = P0, P1, P2; C2 = P3, P4, P5**.
- (ii) **Add C3** → rebalance → 2 each: **C1 = P0, P1; C2 = P2, P3; C3 = P4, P5**.
- **Issue:** a rebalance is triggered; with the (eager) default all consumers **stop and revoke everything** (brief unavailability) and even C2 loses P4/P5 and gets P2/P3 — many partitions move; any **local state/cache** tied to partitions is lost. With a different partition count Range can also be **uneven** (e.g. 7 partitions → 3/2/2 in favour of earlier consumers) and across many topics the first consumer is overloaded.
- **Remedies:** use **Sticky / CooperativeSticky** (minimal movement, no stop-the-world), RoundRobin for even spread (would give C1 = P0,P3; C2 = P1,P4; C3 = P2,P5), and/or **static membership**.

C6. Commits and offsets (asked in SEE 2022)

→ Commits & Offsets · Part 2 p.73 → Auto · Part 2 p.75 → commitSync · Part 2 p.77 → commitAsync · Part 2 p.78 → Specific offset · Part 2 p.81

- Kafka does **not** track per-message acknowledgements like JMS queues. Consumers track their **own position**. Updating the current position = an **offset commit**.
- A consumer commits by sending a message to Kafka, which writes it to the special `__consumer_offsets` topic (per partition). After a rebalance, a consumer reads the **latest committed offset** of each partition and continues from there.
- Kafka does **not commit records individually**: committing offset N implies **everything before N** was processed. Committed offset = **offset of the next message to read** (last processed + 1).

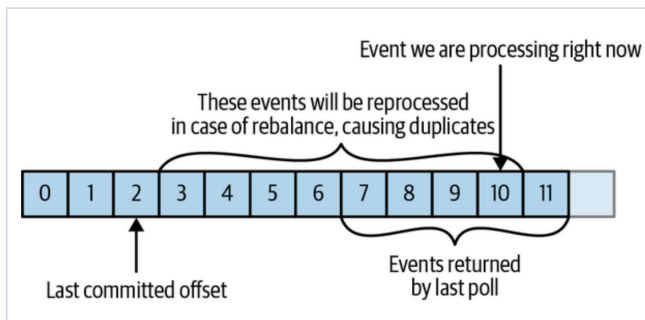


Fig 4-8. Committed offset < last processed → messages reprocessed (duplicates)

→ See in book · Part 2 p.74

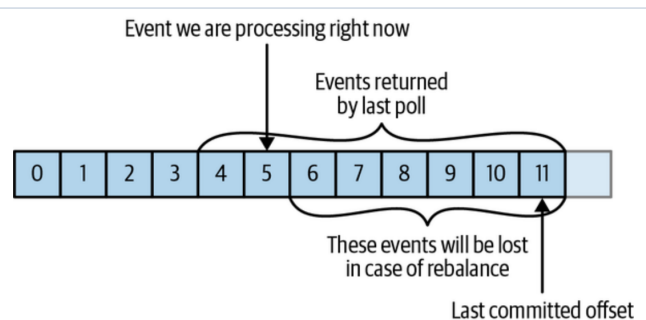


Fig 4-9. Committed offset > last processed → messages missed (lost)

→ See in book · Part 2 p.75

★ IMPORTANT Why it matters

- **Committed < processed** → messages between them are **processed twice** (duplicates).
- **Committed > processed** → messages between them are **missed** (data loss for the group).

Strategy	How	Pros	Cons
Automatic commit	<code>enable.auto.commit=true</code> ; every 5 s (auto.commit.interval.ms) the next poll() commits the last offset returned by the previous poll	Simplest	Crash 3 s after a commit → 3 s of events processed twice . Cannot eliminate duplicates. Must process all records before calling poll() again

Strategy	How	Pros	Cons
Commit current offset (commitSync)	<code>enable.auto.commit=false</code> ; call <code>commitSync()</code> after the batch is processed. Retries until success or unrecoverable error	Most reliable	App is blocked until the broker replies → lower throughput. Commit before finishing = risk of loss
Asynchronous commit (commitAsync)	Send commit and continue; optional <code>OffsetCommitCallback</code>	Higher throughput	No retry (a retry of offset 2000 could overwrite committed 3000 → duplicates). Use a sequence number if retrying
Combine sync + async	<code>commitAsync()</code> in the loop, <code>commitSync()</code> in finally before close/rebalance	Fast and safe at shutdown	Slightly more code
Commit specific offsets	Pass a <code>Map<TopicPartition, OffsetAndMetadata></code> e.g. every 1000 records; commit offset+1	Commit mid-batch	Must track offsets for every partition

🔪 EXAM TIP Exam tip — "any two offset commit strategies"

- Pick **automatic** and **manual synchronous** (or async). For each: how it is set, one advantage, one drawback (duplicates vs blocking). Draw Fig 4-8 / 4-9 to show duplicates/missed messages.

```
while (true) {
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String,String> r : records) { process(r); }
    consumer.commitAsync();           // fast; next commit acts as retry
}
...
finally { consumer.commitSync(); consumer.close(); } // last commit must succeed
```

C7. Rebalance listeners, seeking, clean exit

→ Rebalance listener · Part 2 p.83 → Seek · Part 2 p.87 → Exit / wakeup · Part 2 p.88

ConsumerRebalanceListener method	When called / what to do
<code>onPartitionsAssigned</code>	After reassignment, before consuming. Load state, seek to offsets (must return within max.poll timeout)
<code>onPartitionsRevoked</code>	When giving up partitions (eager: before rebalance starts after consumption stops; cooperative: at end, only for the revoked subset). Commit offsets here , close resources
<code>onPartitionsLost</code>	Cooperative only, exceptional case: partitions were taken without being revoked first ; clean up carefully (new owner may already hold state)

- Pass the listener to `subscribe(topics, listener)`. Typical use: `commitSync(currentOffsets)` in `onPartitionsRevoked`.
- **Consuming from specific offsets:** `seekToBeginning()`, `seekToEnd()`, `seek(tp, offset)`; use `offsetsForTimes()` (timestamp index) to rewind to e.g. one hour ago — useful to recover a lost file or skip ahead when lagging.
- **Clean exit:** another thread (e.g. ShutdownHook) calls `consumer.wakeup()` → `poll()` throws **WakeupException** (ignore it) → finally call `consumer.close()`, which **commits offsets** and tells the coordinator it is leaving → **immediate rebalance**.

C8. Deserializers

→ Deserializers · Part 2 p.90 → Custom · Part 2 p.92 → Avro deserializer · Part 2 p.94

- Mirror of serializers: **bytes** → **objects**. The deserializer **must match** the serializer (IntSerializer + StringDeserializer fails).
- **Custom deserializer** (implements `Deserializer<T>`) **tightly couples** producers and consumers; fragile — **not recommended**. Use JSON / Thrift / Protobuf / Avro.
- **Avro:** set `value.deserializer=KafkaAvroDeserializer`, `schema.registry.url`, and `specific.avro.reader=true` to get the generated class. Incompatibilities are caught with clear errors instead of debugging raw bytes.

C9. Standalone consumer (no group) + exam-ready consumer program

→ Standalone consumer · Part 2 p.96

- Use when a single consumer must read **all partitions** (or specific ones) — no groups, no rebalances. **Do not subscribe; instead assign()** partitions (get them via `partitionsFor(topic)`). A consumer either **subscribes OR assigns**, not both.
- `group.id` is still needed to commit offsets. Trade-off: the consumer is **not notified when partitions are added** — recheck `partitionsFor()` periodically or restart.

CIE-1 / CIE-2 2022: consume a **JSON** topic `sales_data` and auto-convert to a Python dictionary (kafka-python):

```
from kafka import KafkaConsumer
import json

def json_deserializer(data):          # bytes -> dict
    return json.loads(data.decode("utf-8"))

consumer = KafkaConsumer(
    "sales_data",                      # topic
    bootstrap_servers="localhost:9092",
    group_id="sales_group",           # consumer group
    auto_offset_reset="earliest",     # no committed offset -> from beginning
    enable_auto_commit=True,          # auto commit every 5 s
    value_deserializer=json_deserializer, # automatic conversion
)
print("Listening on 'sales_data' ...")
for message in consumer:              # the poll loop
    print(f"Topic: {message.topic}, Partition: {message.partition}, Offset: {message.offset}")
    print("Value (dict):", message.value)
```

EXAM TIP Say in the answer

- `group_id` → consumer group; `value_deserializer` → bytes → dict; `auto_offset_reset` → where to start; the for-loop is the poll loop; use `enable_auto_commit=False` + `consumer.commit()` for tighter control.

Part D — Rapid Revision

Cheat sheets · model answers · flash Q&A

D1. One-page cheat sheet

Concept	One-line answer
Kafka	Distributed commit-log / streaming platform; durable, ordered, replicated, scalable pub/sub
Message / key / batch	Byte array / optional byte array deciding the partition / group of messages for one topic-partition
Topic / partition	Category / one append-only log; ordering only within a partition
Offset	Increasing id of a message within its partition; consumer's bookmark
Broker / controller	One Kafka server / broker elected to manage partition assignment & failures
Leader / follower	Partition owner (producers write here) / replica copy that can take over
Retention	Time (7 d) or size (1 GB) based deletion; log compaction keeps last per key
MirrorMaker	Consumer + producer copying data between clusters / datacentres
Consumer group	Consumers sharing a topic: 1 partition = 1 consumer in a group; each group gets all messages
Rebalance	Reassigning partitions on join / leave / crash / partition change
Group coordinator / leader	Broker tracking heartbeats / first consumer, computes assignment
Heartbeat	Liveness signal; miss session.timeout.ms → dead → rebalance
Idempotent producer	Sequence numbers → broker discards duplicate retries
Schema Registry	Stores Avro schemas; records carry only the schema id
__consumer_offsets	Internal topic storing committed offsets

D2. Producer vs Consumer at a glance

	Producer	Consumer
Object	KafkaProducer / ProducerRecord	KafkaConsumer / ConsumerRecord
Mandatory config	bootstrap.servers, key.serializer, value.serializer	bootstrap.servers, key.deserializer, value.deserializer (+ group.id)
Converts	Object → bytes (serializer)	Bytes → object (deserializer)
Chooses partition	Partitioner (hash of key / sticky / custom)	Assignor (Range, RoundRobin, Sticky, CoopSticky) via group leader
Reliability knob	acks, retries/delivery.timeout.ms, idempotence	Offset commit strategy, auto.offset.reset
Throughput knob	linger.ms, batch.size, compression.type	fetch.min.bytes, max.poll.records, more consumers/partitions
Talks to	Partition leader	Leader or a follower (client.rack)
Loop	send() (fire-forget / sync / async+callback)	poll() loop + commit

D3. Model answers (outline)

★ IMPORTANT Q. What happens if a broker crashes? (CIE-1, CIE-2, SEE)

- Controller detects failure; for each partition led by that broker, a **follower from the in-sync replicas is elected leader** (needs replication factor > 1).
- Producers get *not-leader* errors, refresh metadata and **retry** (**delivery.timeout.ms** should exceed the ~30 s election time) — with acks=all no acknowledged data is lost.
- Consumers reconnect to new leaders and resume from the **last committed offset**; messages are replayed from the log (some duplicates possible).
- Retention lets the cluster keep data while the broker is repaired; capacity restored by adding a broker online.

★ IMPORTANT Q. Explain consumers and consumer groups with an example (SEE 2022)

- Define group (consumers sharing **group.id**, each partition owned by one member).
- Topic T1 with 4 partitions: C1 alone → all 4; add C2 → 2 + 2; 4 consumers → 1 each; 5th consumer idle. (Draw Figs 4-1 to 4-4.)
- Second application = **new group G2** → also receives all messages (Fig 4-5).
- Failure/join → **rebalance**, heartbeats and coordinator; offsets stored in `__consumer_offsets` so work resumes.

★ **IMPORTANT Q. Reasons to choose Kafka / data ecosystem (Model QP)**

- List six: multiple producers, multiple consumers, disk-based retention, scalable, high performance, platform features (Connect, Streams).
- Ecosystem: producers → Kafka (circulatory system) → consumers/stream processors/Hadoop/search; Schema Registry decouples them; draw Fig 1-9.

D4. Flash Q&A; — cover the right column and test yourself

#	Question	Answer
1	Unit of data in Kafka?	A message (byte array) with optional key.
2	Is ordering guaranteed across a topic?	No — only within a partition.
3	What does a producer need at minimum?	<code>bootstrap.servers</code> , <code>key.serializer</code> , <code>value.serializer</code> .
4	Name the 3 send methods.	Fire-and-forget, synchronous, asynchronous (callback).
5	<code>acks=0 / 1 / all</code> ?	No wait / leader only / all in-sync replicas.
6	How to avoid duplicate writes on retry?	<code>enable.idempotence=true</code> (sequence numbers).
7	Same key always same partition — until?	The partition count changes.
8	What does <code>linger.ms</code> do?	Waits to fill batches: +latency, +throughput.
9	Where do Avro schemas live in Kafka setups?	In a Schema Registry; messages carry the schema id.
10	Who can a producer write to?	The partition leader only.
11	3 quota types?	Produce, consume, request.
12	More consumers than partitions?	Extra consumers are idle.
13	How does a second app get all messages?	Give it its own consumer group.
14	Eager vs cooperative rebalance?	Revoke all & stop the world vs move only a subset incrementally.
15	Default assignor?	RangeAssignor.
16	Heartbeat vs session timeout ratio?	<code>heartbeat.interval.ms</code> ≈ 1/3 of <code>session.timeout.ms</code> .
17	What if <code>poll()</code> isn't called for <code>max.poll.interval.ms</code> ?	Consumer is considered dead; rebalance.
18	<code>auto.offset.reset</code> values?	latest (default), earliest, none.
19	Default auto-commit interval?	5 seconds (<code>auto.commit.interval.ms</code>).
20	Committed offset > processed offset?	Messages missed (lost).
21	Committed offset < processed offset?	Messages reprocessed (duplicates).
22	<code>commitSync</code> vs <code>commitAsync</code> ?	Blocks & retries vs non-blocking & no retry.
23	Where are offsets stored?	Internal topic <code>__consumer_offsets</code> .
24	Safe way to stop a consumer?	<code>wakeup()</code> from another thread, then <code>close()</code> .
25	<code>assign()</code> vs <code>subscribe()</code> ?	Manual partitions, no group/rebalance vs group-managed.
26	Four goals of Kafka at LinkedIn?	Decouple (push-pull), persistence, high throughput, horizontal scaling.
27	Who created Kafka?	Jay Kreps, Neha Narkhede, Jun Rao at LinkedIn; Confluent founded 2014.

D5. Jump back to the book

Everything in blue opens the textbook at the right page:

Part 1 (Ch.1)

→ Pub/sub · Part 1 p.2 → Messages & batches · Part 1 p.6 → Topics & partitions · Part 1 p.8 → Producers & consumers · Part 1 p.9 → Brokers & clusters · Part 1 p.11 → Why Kafka · Part 1 p.14 → Ecosystem · Part 1 p.17 → Use cases · Part 1 p.18

→ Origin · Part 1 p.21

Part 2 – Producers (Ch.3)

→ Overview · Part 2 p.2 → Constructing · Part 2 p.5 → Sending · Part 2 p.8 → Config · Part 2 p.12 → Serializers · Part 2 p.23
→ Avro · Part 2 p.26 → Partitions · Part 2 p.33 → Interceptors · Part 2 p.37 → Quotas · Part 2 p.40

Part 2 – Consumers (Ch.4)

→ Groups · Part 2 p.45 → Rebalance · Part 2 p.53 → Create / subscribe · Part 2 p.58 → Poll loop · Part 2 p.60 → Config ·
Part 2 p.63 → Commits · Part 2 p.73 → Listeners · Part 2 p.83 → Deserializers · Part 2 p.90 → Standalone · Part 2 p.96

EXAM TIP

- Revise in this order tonight: **Part D flash Q&A** → **IMPORTANT boxes** in A, B, C → the two code programs → figures. Good luck!